

大模型推训优化

先 profile 再优化：small-M Router GEMM 与 CPU-GPU bubble

- Router GEMM
- Profiling
- GPU-local state
- CUDA Graph

岗位能力画像

岗位画像：考察从实际 shape 出发的选核能力，以及把 Kernel 速度、状态正确性和端到端收益分层说明的能力。

高频深挖：题目背后的判断

small-M 为什么特别难？

固定大 tile 在小工作量下会产生无效计算；必须让后端随 shape、dtype 和图约束分流。

shape-aware selector + fallback。

Nsys 先看什么？

找 launch gap、同步、D2H/H2D、通信等待和 stream overlap，而不是先盯一个 kernel。

时间线先分类，再进单 kernel。

为什么 CPU 小逻辑也会拖 decode？

历史状态往返会形成 bubble；优化后仍要覆盖 mixed batch、reorder 和恢复语义。

性能路径是状态机的一部分。

Graph 能否替代融合？

Graph 降低 host submission；fusion 减少 launch 和中间访存，先判断瓶颈再选。

两者解决不同层面的开销。

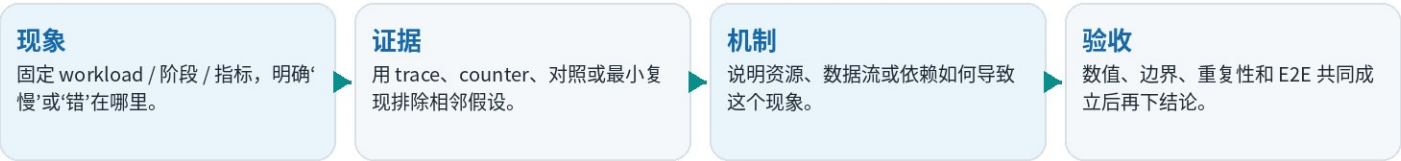
项目叙事的通用深挖链



答题提醒

不要从‘用了什么库/指令’开始；先固定工作负载、指标和约束，再说明数据流/资源/依赖，最后给出正确性、性能和反例边界。

把一句结论变成可复验的事实



一题答透：时间线读法：把 bubble 变成一个可检验的假设

问题 “decode 中出现空洞，怎样判断是 launch、同步、数据往返还是 kernel 本体？”

1. 按阶段切片

把 prefill、decode、采样和状态更新分开；不同阶段不能共用一个平均值。

2. Nsys 分类

看 CPU launch gap、同步、双向拷贝、collective 与同 stream/跨 stream overlap。

3. NCU 深钻

只针对热点 kernel 看 SM active、DRAM/L2、eligible warps、stall 与资源预算。

4. 单变量验收

每个改动对应一个假设：少一次往返、少一次 launch、少一个无效 tile 或多一条独立链。

把推理链画出来



常见误区

× 把联合优化收益都归因给一个 kernel。

× TP>1 沿用单卡 fused 路径，却没有确定性/状态矩阵。

× 只用平均吞吐，忽略不同 batch、shape 和阶段。

术语速懂 | 面试怎么答

术语速懂

Nsys → NCU

Nsys 先归类时间线上的 launch、同步、拷贝和通信；NCU 再深挖确认热点的资源与 stall。

small-M Router GEMM

M 很小时固定大 tile 会产生无效线程与 mask；需依 shape、dtype、layout 和图约束分流。

面试中怎么答

- ① 先按阶段用 Nsys 给 bubble 分类，不把平均吞吐当结论。
- ② 再对热点用 NCU 验证一个可删除成本，而非同时改多处。
- ③ 最后用 shape 矩阵、状态路径和 E2E 回归闭环。

白板 / 手撕：白板题：Row-Max 的优化阶梯

题面 给 [M,N] 矩阵，每行求最大值。请从可运行 baseline 逐步讲到 warp/block reduction。

CUDA C++ · row-wise max (one block per row)

核心实现

```
01 __device__ float warp_max(float v) {
02     for (int d = 16; d; d >>= 1)
03         v = fmaxf(v, __shfl_down_sync(0xffffffff, v, d));
04     return v;
05 }
06 __global__ void row_max(const float* x, float* y, int M, int N) {
07     int r = blockIdx.x, t = threadIdx.x; if (r >= M) return;
08     float v = -CUDART_INF_F;
09     for (int c = t; c < N; c += blockDim.x)
10         v = fmaxf(v, x[r * N + c]);
11     v = warp_max(v);
12     __shared__ float partial[32];
13     if ((t & 31) == 0) partial[t >> 5] = v;
14     __syncthreads();
15     if (t < 32) v = t < (blockDim.x + 31) / 32
16         ? partial[t] : -CUDART_INF_F;
17     if (t < 32) v = warp_max(v);
18     if (t == 0) y[r] = v;
19 }
```

写代码时怎么讲

1. 一个 block 负责一行，线程按 tid + t·blockDim 读 N，寄存器保存 local max。
2. warp 内用 shuffle 降同步；多 warp 时把 warp result 写 shared，再做最后归约。
3. 检查 coalescing、N 的尾部 mask、block size 与 M 很小时的并发不足。
4. 先写测试：随机、全负、-inf、NaN、非整除 N。

实现重点

- 核心：register local max → warp shuffle → block final reduce。
- c < N 是 tail mask；NaN 语义要主动说明。
- M 很小时并发不足，需评估多 row/block 或 persistent 策略。

边界、测试与追问

最少要说明的验证

- 核心：register local max → warp shuffle → block final reduce。
- c < N 是 tail mask；NaN 语义要主动说明。
- M 很小时并发不足，需评估多 row/block 或 persistent 策略。

面试继续追问

- 什么时候需要多 block/row？
- 如何证明优化没有改变确定性要求？
- binary search + feasibility 怎样最小化最大分段和？